

# Implementation Plan: Near-Term Trend Constraint via Bayesian Predictive Synthesis (Revised)

## Overview

This document specifies the implementation of a near-term trend constraint module that combines a non-parametric trend forecast with the rate-and-state model projection using Bayesian Predictive Synthesis (BPS). The synthesis hyperparameters are calibrated from leave-future-out cross-validation on the satellite-era record. Their posterior uncertainty is propagated through the combined predictive, producing credible intervals that account for uncertainty in both the individual forecasts and the combination itself.

Read the companion LaTeX document `near_term_trend_constraint.tex` for the full mathematical derivation before implementing.

---

## Key Design Decisions

- Bayesian Predictive Synthesis, not MSE-optimal weighting.** The combination uses BPS (McAlinn & West, 2019) with a structured latent weight process calibrated from cross-validation. This replaces the MSE-optimal point-forecast weighting, which produced systematically overconfident predictive intervals: the linear-combination ensemble spread was  $w^2 \cdot \text{Var}_{\text{trend}} + (1-w)^2 \cdot \text{Var}_{\text{model}}$ , strictly narrower than either individual ensemble for  $0 < w < 1$ . BPS produces a properly calibrated mixture predictive.
- Synthesis hyperparameters calibrated, not assumed.** The previous approach pegged  $\kappa$  to the model's own  $\partial r / \partial T$  — a circular estimate where model overconfidence in its rate-temperature sensitivity accelerated its own dominance. The BPS framework calibrates the synthesis hyperparameters via leave-future-out cross-validation, replacing the assumed  $\kappa$  with a data-informed posterior. The posterior reports which aspects of the combination are identified by data ( $\phi_0$ ,  $\Delta t^*_{\text{low}}$ ) and which remain prior-dominated ( $\kappa_{\text{net}}$  at large  $\Delta T$ ).
- No double-counting of acceleration-estimation variance.** The previous  $\kappa_t^{\text{eff}} = \kappa_t + \frac{1}{4}\sigma^2_{\text{rdot}} \cdot dt^4$  construction double-counted: the  $\frac{1}{4}\sigma^2_{\text{rdot}} \cdot dt^4$  term was already captured by  $\text{Var}[H_{\text{trend}} \text{ samples}]$  (computed via Monte Carlo from the WLS posterior), and then added again through  $\kappa_t^{\text{eff}}$ . The corrected architecture uses the trend ensemble

variance directly. The  $\kappa_t^{\text{eff}}$  concept is eliminated.

4. **AIS dynamic component exempt from trend suppression.** The BPS synthesis operates on total GMSL minus the AIS dynamic contribution. AIS dynamics are added back to the combined projection without passing through the trend-model weighting. This prevents the trend constraint from vetoing near-term WAIS instability scenarios.
  5. **Mixture predictive for uncertainty reporting; stochastic-transition trajectories for coherence.** The combined predictive is a mixture distribution (correctly calibrated marginals in the  $\epsilon \rightarrow 0$  limit; see §Combination for the finite- $\epsilon$  approximation). Temporally coherent trajectories for rate-of-change analysis and counterfactual queries use sample-specific synthesis weights with a smooth stochastic transition, preceded by quantile-matching to minimize rate discontinuities.
  6. **Shared-data covariance handled by construction.** In the BPS framework, the shared-data covariance is handled implicitly through the cross-validation calibration: both agents are evaluated against the same held-out observations, and the synthesis log-likelihood captures whatever covariance structure exists in their predictions without requiring an explicit covariance computation.
  7. **GIA and AIS-subtraction uncertainty propagated.** The WLS rate estimate depends on the GIA correction and the AIS subtraction. An additional variance  $\sigma^2_{\text{GIA}} = (0.15 \text{ mm/yr})^2$  is added to  $\text{Var}(\hat{r})$  in the propagated  $3 \times 3$  covariance. A time-dependent AIS subtraction uncertainty  $\sigma_{\text{AIS}}(t)$  is added in quadrature to the per-observation altimetry uncertainty, with larger values for the pre-GRACE era (1993–2002).
  8. **Reparameterized in terms of the low-emission crossover time.** The sampled synthesis hyperparameters are  $\psi = (\phi_0, \kappa_{\text{net}}, \Delta t^*_{\text{low}})$ , where  $\Delta t^*_{\text{low}}$  is the crossover lead time under zero forcing departure. The derived  $\kappa_t = \phi_0 / (\Delta t^*_{\text{low}})^4$ . This makes prior specification transparent: one directly specifies how long the trend is trusted under constant forcing.
  9.  **$\lambda$  (model structural bias) removed.** The previous MSE-space parameter  $\lambda$  for the model's structural bias growth had no computational role in BPS: the model agent density  $h_{\text{model}}$  is the empirical ensemble distribution (which captures parametric uncertainty but not  $\lambda$ ), and the weight process uses  $\kappa_{\text{net}}$  independently.  $\lambda$  is removed from the framework. The sensitivity diagnostic comparing  $\Delta T^2 \cdot dt^2$  vs  $\Delta T^4 \cdot dt^2$  scaling is reframed directly in terms of the  $\kappa_{\text{net}}$  functional form.
  10. **Module placement under diagnostics/.** The BPS synthesis is a post-processing module, not a physical component. Placing it under components/ would create confusion about whether it participates in the budget constraint (it does not).
-

# File Structure

All new code goes under the existing project layout:

```
gmslr_framework/
├─ diagnostics/
│   └─ trend_combination/
│       ├── __init__.py
│       ├── wls_trend_model.py          # WLS quadratic fit to satellite-era GMSL
│       ├── trend_projection.py         # Trend projection ensemble generation
│       ├── bps_synthesis.py           # BPS weight process, calibration, posterior
│       ├── combination.py             # Mixture predictive, stochastic-transition
│       ├── crossover_diagnostics.py    # Crossover computation and reporting
│       └─ tests/
│           ├── test_wls_trend_model.py
│           ├── test_trend_projection.py
│           ├── test_bps_synthesis.py
│           ├── test_combination.py
│           └─ test_crossover_diagnostics.py
└─ utils/
    ├── temperature_departure.py        #  $\Delta T(t)$  computation from SSP trajectories
    └─ tests/
        └─ test_temperature_departure.py
```

---

## Module 1: `utils/temperature_departure.py`

**Purpose:** Compute the temperature departure  $\Delta T(t)$  from the calibration domain for arbitrary SSP trajectories.

## Functions

```
def compute_T_cal_max(T_cal: np.ndarray) -> float:
    """
    Maximum GMST in the calibration window.

    Parameters
    -----
    T_cal : np.ndarray, shape (n_cal,)
        Annual GMST anomalies during the calibration period.

    Returns
    -----
    T_max_cal : float
```

```

        Maximum observed GMST (°C relative to baseline).
    """

def compute_delta_T(
    T_scenario: np.ndarray,
    T_max_cal: float,
    overshoot_mode: bool = True,
) -> np.ndarray:
    """
    Temperature departure from the calibration domain.

    Parameters
    -----
    T_scenario : np.ndarray, shape (n_proj,)
        GMST trajectory for a given SSP, covering [t_end+1, ..., t_end+n_proj].
    T_max_cal : float
        Maximum GMST in the calibration window.
    overshoot_mode : bool
        If True, use cumulative maximum exceedance:
             $\Delta T(t) = \max_{s \in [t_{\text{end}}, t]} \max(0, T(s) - T_{\text{max\_cal}})$ 
        If False, use instantaneous exceedance:
             $\Delta T(t) = \max(0, T(t) - T_{\text{max\_cal}})$ 

    Returns
    -----
    delta_T : np.ndarray, shape (n_proj,)
        Temperature departure at each projection time step. Units: °C.
    """

```

## Implementation notes

- `overshoot_mode=True` is the default and recommended setting.
- Both functions are pure numpy.
- `T_scenario` and `T_cal` must use the same baseline/reference period. The caller is responsible for ensuring this.

## Tests

1. **Zero departure:** If  $T_{\text{scenario}} \leq T_{\text{max\_cal}}$  everywhere, `delta_T` is identically zero.
2. **Monotone increasing:** If `T_scenario` is monotonically increasing, both modes give  $\max(0, T_{\text{scenario}} - T_{\text{max\_cal}})$ .
3. **Overshoot:** If `T_scenario` = [1.5, 1.8, 1.6, 1.4] and `T_max_cal` = 1.2, then:

- Instantaneous: [0.3, 0.6, 0.4, 0.2]
- Overshoot mode: [0.3, 0.6, 0.6, 0.6]

---

## Module 2: `diagnostics/trend_combination/wls_trend_model.py`

**Purpose:** Fit a weighted least-squares quadratic to satellite-era GMSL altimetry and extract the posterior on the level, rate, and acceleration at the end of the calibration window. Includes GIA uncertainty inflation and AIS subtraction uncertainty.

### Dependencies

- `statsmodels` (WLS regression)
- `numpy`, `scipy`

### Class

```
class WLS_TrendModel:
    """
    Weighted least-squares quadratic model for satellite-era GMSL
    trend estimation.

    Fits  $H(t) = \alpha + \beta \cdot (t - t_{\text{ref}}) + \gamma \cdot (t - t_{\text{ref}})^2$  to annual-mean
    GMSL from the satellite era (1993- $t_{\text{end}}$ ), then extracts the
    posterior on the rate ( $dH/dt$ ) and acceleration ( $d^2H/dt^2$ ) at  $t_{\text{end}}$ .
    """

    def __init__(
        self,
        satellite_start: float = 1993.0,
        sigma_gia: float = 0.15,
        fix_scale: bool = True,
    ):
        """
        Parameters
        -----
        satellite_start : float
            Start year of the satellite altimetry era.
        sigma_gia : float
            GIA inter-model rate uncertainty (mm/yr). Added in quadrature
            to  $\text{Var}(\hat{r})$  in the propagated covariance. Default 0.15 mm/yr.
        fix_scale : bool
            If True, fix the residual scale at  $\sigma^2 = 1.0$  (trusted
```

```

        observation uncertainties). If False, estimate  $\sigma^2$  from
        weighted residuals.
    """

def fit(
    self,
    years: np.ndarray,
    gmsl: np.ndarray,
    gmsl_uncertainty: np.ndarray | None = None,
    ais_subtraction_uncertainty: np.ndarray | None = None,
) -> "WLSTrendModel":
    """
    Fit the WLS quadratic to satellite-era GMSL data.

    Parameters
    -----
    years : np.ndarray, shape (n_obs,)
    gmsl : np.ndarray, shape (n_obs,)
        Observed non-AIS GMSL (mm).
    gmsl_uncertainty : np.ndarray, shape (n_obs,) or None
        Per-observation 1 $\sigma$  altimetry uncertainty (mm). If None, OLS is used.
    ais_subtraction_uncertainty : np.ndarray, shape (n_obs,) or None
        Per-observation 1 $\sigma$  AIS subtraction uncertainty (mm). Added in
        quadrature to gmsl_uncertainty to form effective weights.
        Larger for pre-GRACE era (1993-2002), smaller for GRACE era.
        If None, no AIS subtraction uncertainty is included.
    """

    # Compute effective uncertainty:
    #   sigma_eff = sqrt(gmsl_uncertainty^2 + ais_subtraction_uncertainty^2)
    # Subset to satellite era
    # Center time for numerical stability: t_ref = midpoint
    # Design matrix: [1, dt, dt^2]
    # Fit via statsmodels WLS with weights = 1/sigma_eff^2
    # If fix_scale: use cov_type='fixed scale', cov_kws={'scale': 1.0}
    # Else: use default residual variance estimation

def predict_at_t_end(self, t_end: float) -> dict:
    """
    Extract posterior at t_end for the level, rate, and acceleration.

    The transformation from ( $\alpha$ ,  $\beta$ ,  $\gamma$ ) to (H, r, rdot) at t_end is:
        H   =  $\alpha + \beta \cdot \Delta + \gamma \cdot \Delta^2$       where  $\Delta = t_{\text{end}} - t_{\text{ref}}$ 
        r    =  $\beta + 2\gamma \cdot \Delta$ 
        rdot =  $2\gamma$ 

    The 3x3 covariance is J @  $\Sigma_{\text{params}}$  @ J.T, where J is the Jacobian.
    GIA uncertainty is added to the (1,1) element (Var(r)).
    """

```

Returns

-----

result : dict with keys:

"H\_mean", "H\_std" : float

"r\_mean", "r\_std" : float (r\_std includes GIA inflation)

"rdot\_mean", "rdot\_std" : float

"cov\_r\_rdot" : float

"cov\_3x3" : np.ndarray, shape (3, 3) – includes GIA inflation

"""

```
def sample_derivatives(
```

```
    self, t_end: float, n_samples: int = 10000, seed: int | None = None,
```

```
) -> dict:
```

"""

Draw joint posterior samples of (H, r, rdot) at t\_end.

Samples from the 3D multivariate normal defined by the WLS posterior (with GIA inflation).

Returns

-----

samples : dict with keys "H", "r", "rdot", each shape (n\_samples,)

"""

```
def robustness_checks(
```

```
    self,
```

```
    years: np.ndarray,
```

```
    gmsl: np.ndarray,
```

```
    gmsl_uncertainty: np.ndarray | None,
```

```
    ais_subtraction_uncertainty: np.ndarray | None,
```

```
    t_end: float,
```

```
    short_window_years: int = 20,
```

```
) -> dict:
```

"""

Run cubic test and window-sensitivity robustness checks.

Returns

-----

checks : dict with keys:

"cubic\_coeff" : float

"cubic\_coeff\_se" : float

"cubic\_significant" : bool – True if  $|\delta/\text{se}(\delta)| > 2$

"cubic\_rdot\_at\_tend" : float –  $2\gamma + 6\delta \cdot (t_{\text{end}} - t_{\text{ref}})$

"short\_window\_r" : float

"short\_window\_rdot" : float

"short\_window\_r\_diff\_sigma" : float

```

    "short_window_rdot_diff_sigma" : float
    "window_sensitive" : bool - True if either > 1σ
"""

```

## Tests

1. **Sanity on synthetic data:** Generate  $\text{GMSL} = 50 \cdot t + 3 \cdot t^2 + \text{noise}$ , verify  $r_{\text{mean}} \approx 50 + 6 \cdot t_{\text{end}}$  and  $\text{rdot}_{\text{mean}} \approx 6.0$  within WLS uncertainty.
2. **Covariance correctness:** On synthetic data with known noise, verify  $3 \times 3$  covariance matches empirical bootstrap covariance.
3. **GIA inflation:** Verify that  $r_{\text{std}}$  with  $\text{sigma}_{\text{gia}} > 0$  exceeds  $r_{\text{std}}$  with  $\text{sigma}_{\text{gia}} = 0$  by the expected amount. Verify that  $\text{rdot}_{\text{std}}$  is unchanged.
4. **Scale fixing:** With  $\text{fix}_{\text{scale}} = \text{True}$  and accurate observation uncertainties, parameter covariance should be  $(X^T W X)^{-1}$ . With  $\text{fix}_{\text{scale}} = \text{False}$ , it should be  $\hat{\sigma}^2 (X^T W X)^{-1}$ .
5. **Derivative variance ordering:**  $r_{\text{std}} < \text{rdot}_{\text{std}}$ .
6. **Exact recovery:** With zero noise, the WLS recovers exact coefficients and covariance is zero (within float tolerance).
7. **Statsmodels agreement:** Jacobian-propagated covariance matches reparametrised fit on  $[1, t - t_{\text{end}}, (t - t_{\text{end}})^2]$ .
8. **Cubic robustness:** On pure quadratic data,  $\text{cubic}_{\text{significant}} == \text{False}$ . On data with known cubic, detection works.
9. **Window sensitivity:** On data with stationary acceleration,  $\text{window}_{\text{sensitive}} == \text{False}$ .
10. **AIS subtraction uncertainty:** Verify that adding  $\text{ais}_{\text{subtraction}_{\text{uncertainty}}}$  increases parameter uncertainties. Verify that the effect is larger when the AIS uncertainty is concentrated in the early satellite era (1993–2002).

---

## Module 3: `diagnostics/trend_combination/trend_projection.py`

**Purpose:** Generate trend projection ensembles from the WLS posterior samples.

**Note:** The MSE computation from the previous design is removed. The trend ensemble (which captures estimation variance via Monte Carlo) is the sole output consumed by the BPS synthesis. The MSE-space parameters  $\kappa$  and  $\kappa_t$  are distinct from the logit-space synthesis hyperparameters and are not computed here.



## Functions

```
def trend_projection_ensemble(
    t_proj: np.ndarray,
    t_end: float,
    derivative_samples: dict,
) -> dict:
    """
    Generate trend projection ensemble.

    The trend projection for each sample s is:
         $H_{\text{trend}}(s)(t) = H(s) + r(s) \cdot (t - t_{\text{end}}) + 0.5 \cdot r_{\text{dot}}(s) \cdot (t - t_{\text{end}})^2$ 

    Parameters
    -----
    t_proj : np.ndarray, shape (n_proj,)
    t_end : float
    derivative_samples : dict - output of WLSTrendModel.sample_derivatives()

    Returns
    -----
    result : dict with keys:
        "H_ensemble" : np.ndarray, shape (n_samples, n_proj)
    """
```

## Implementation notes

- The ensemble captures all estimation-uncertainty contributions ( $\sigma_r^2$ ,  $\frac{1}{4}\sigma_{r\text{dot}}^2$ , and all cross-terms) exactly via Monte Carlo, eliminating the need for any analytical MSE decomposition.

## Tests

1. **Deterministic:** For fixed (H, r, rdot) samples at known values, verify `H_ensemble` matches the quadratic formula.
2. **Variance growth:** Verify that `VarH_ensemble` grows approximately as  $\sigma_r^2 \cdot dt^2 + \frac{1}{4}\sigma_{r\text{dot}}^2 \cdot dt^4$  (to within cross-term corrections).
3. **Shape consistency:** Output shape matches (n\_samples, n\_proj).

---

## Module 4: `diagnostics/trend_combination/bps_synthesis.py`

**Purpose:** Implement the Bayesian Predictive Synthesis framework: weight process, leave-

future-out cross-validation, posterior sampling.

## Dependencies

- `numpy`, `scipy`
- `emcee` (MCMC sampler)

## Functions and Classes

```
from dataclasses import dataclass

@dataclass
class SynthesisHyperparameters:
    """
    Synthesis hyperparameters  $\psi = (\phi_0, \kappa_{\text{net}}, \Delta t^*_{\text{low}})$ .

    The derived acceleration-persistence coefficient is:
     $\kappa_t = \phi_0 / (\Delta t^*_{\text{low}})^4$ 
    """
    phi_0: float          # Initial logit-weight (positive: trend favored)
    kappa_net: float       # Net forcing-departure degradation (logit-space)
    dt_low: float          # Low-emission crossover time (years)

    @property
    def kappa_t(self) -> float:
        """Derived acceleration-persistence coefficient."""
        return self.phi_0 / self.dt_low**4

def compute_logit_weight(
    dt: np.ndarray,
    delta_T: np.ndarray,
    psi: SynthesisHyperparameters,
) -> np.ndarray:
    """
    Compute the logit of the synthesis weight  $\phi(t; \psi)$ .

    
$$\phi(t) = \phi_0 - \kappa_{\text{net}} \cdot \Delta T(t)^2 \cdot (t - t_{\text{end}})^2 - \phi_0 / (\Delta t^*_{\text{low}})^4 \cdot (t - t_{\text{end}})^4$$


    Parameters
    -----
    dt : np.ndarray, shape (n_proj,)
        Lead times:  $t - t_{\text{end}}$ .
    delta_T : np.ndarray, shape (n_proj,)
        Temperature departure at each projection time.
```

```

psi : SynthesisHyperparameters

Returns
-----
phi : np.ndarray, shape (n_proj,)
"""

def compute_synthesis_weight(
    dt: np.ndarray,
    delta_T: np.ndarray,
    psi: SynthesisHyperparameters,
) -> np.ndarray:
    """
    Compute  $w(t; \psi) = \sigma(\phi(t; \psi))$ .

    Returns
    -----
    w : np.ndarray, shape (n_proj,)
        Synthesis weight on trend agent, in  $[0, 1]$ .
    """

def synthesis_log_likelihood(
    psi: SynthesisHyperparameters,
    cv_data: list[dict],
) -> float:
    """
    Compute the synthesis log-likelihood from cross-validation data.


$$\ell(\psi) = \sum_{j, \Delta t} \log[w(\Delta t, \Delta T; \psi) \cdot h_{\text{trend}}(H_{\text{obs}}) + (1-w) \cdot h_{\text{model}}(H_{\text{obs}})]$$


    Parameters
    -----
    psi : SynthesisHyperparameters
    cv_data : list of dict
        Each dict corresponds to one holdout endpoint  $t_h$  and contains:
        - "dt" : np.ndarray – lead times
        - "delta_T" : np.ndarray – temperature departures
        - "H_obs" : np.ndarray – observed GMSL at held-out times
        - "trend_log_density" : np.ndarray –  $\log h_{\text{trend}}(H_{\text{obs}})$  at each lead
        - "model_log_density" : np.ndarray –  $\log h_{\text{model}}(H_{\text{obs}})$  at each lead

    Returns
    -----
    loglik : float
    """

```

```

def synthesis_log_prior(psi: SynthesisHyperparameters) -> float:
    """
    Compute the log prior on  $\psi$ .

     $\phi_0$       ~ TruncNormal(2.5, 0.52, lower=0)
     $\kappa_{\text{net}}$  ~ LogNormal(log(0.05), 0.6)
     $\Delta t^*_{\text{low}}$  ~ LogNormal(log(20), 0.35)

    Returns
    -----
    logprior : float (-inf if outside support)
    """

def synthesis_log_posterior(
    psi: SynthesisHyperparameters,
    cv_data: list[dict],
) -> float:
    """Log posterior: log_prior + log_likelihood."""

def prior_predictive_check(
    delta_T_trajectories: dict[str, np.ndarray],
    dt: np.ndarray,
    n_prior_samples: int = 1000,
    seed: int | None = None,
) -> dict:
    """
    Mandatory prior predictive check.

    Draws  $n_{\text{prior\_samples}}$  from the joint prior on  $\psi$  and computes
     $w(t; \psi)$  for each sample using the provided  $\Delta T$  trajectories.

    Parameters
    -----
    delta_T_trajectories : dict mapping SSP name to  $\Delta T$  array
    dt : np.ndarray — lead times
    n_prior_samples : int
    seed : int or None

    Returns
    -----
    result : dict with keys:
        "w_prior_samples" : dict[str, np.ndarray], shape (n_prior_samples, n_proj)
        Per-SSP prior weight ensembles.
    """

```

```

"w_median" : dict[str, np.ndarray], shape (n_proj,)
"w_5" : dict[str, np.ndarray]
"w_95" : dict[str, np.ndarray]
"checks_passed" : bool
    True if all four prior predictive checks pass:
    - 5 yr lead, all SSPs: median w > 0.8
    - 10 yr lead, SSP2-4.5: median w in [0.3, 0.8]
    - 20 yr lead, SSP2-4.5: median w < 0.3
    - 20 yr lead, SSP1-2.6 ( $\Delta T \approx 0$ ): median w in [0.3, 0.7]
"""

```

```

class BPSSynthesis:

```

```

    """
    Bayesian Predictive Synthesis for trend-model combination.

    Orchestrates the full BPS workflow: prior predictive check,
    cross-validation data construction, posterior sampling, and
    weight computation.
    """

    def __init__(
        self,
        n_walkers: int = 16,
        n_warmup: int = 2000,
        n_samples: int = 5000,
        seed: int | None = None,
    ):
        """
        Parameters
        -----
        n_walkers : int
            Number of emcee walkers.
        n_warmup : int
            Number of warm-up (burn-in) steps per walker.
        n_samples : int
            Number of sampling steps per walker (after warm-up).
        seed : int or None
        """

    def build_cv_data(
        self,
        years: np.ndarray,
        gmsl: np.ndarray,
        gmsl_uncertainty: np.ndarray | None,
        ais_subtraction_uncertainty: np.ndarray | None,
        T_obs: np.ndarray,
    )

```

```

    model_refit_fn: callable,
    holdout_endpoints: np.ndarray | None = None,
    min_holdout: int = 5,
) -> list[dict]:
    """
    Construct leave-future-out cross-validation data.

    For each holdout endpoint t_h:
    1. Fit WLS to [satellite_start, t_h]
    2. Call model_refit_fn(t_h) to get model posterior predictive
    3. Evaluate trend and model densities at held-out observations

    Parameters
    -----
    years, gmsl, gmsl_uncertainty, ais_subtraction_uncertainty :
        Observation arrays (AIS subtraction uncertainty propagated
        into WLS weights).
    T_obs : np.ndarray
        observed GMST trajectory
    model_refit_fn : callable
        Function that takes t_h and returns a dict:
        {
            "H_predictive_mean": np.ndarray,
            "H_predictive_std": np.ndarray,
        }
    holdout_endpoints : np.ndarray or None
        If None, use default: t_h ∈ {2003, 2005, ..., t_end - 5}.
    min_holdout : int
        Minimum number of held-out years required.

    Returns
    -----
    cv_data : list[dict]
    """

def fit(self, cv_data: list[dict]) -> "BPSSynthesis":
    """
    Sample the posterior on  $\psi$  via emcee.

    Initialises walkers near the prior mode, runs warm-up,
    then samples. Stores posterior samples.

    The sampler operates in transformed space:
    - log( $\kappa_{\text{net}}$ ) sampled directly (ensuring positivity)
    - log( $\Delta t^*_{\text{low}}$ ) sampled directly (ensuring positivity)
    -  $\varphi_0$  sampled with reflecting boundary at 0

    Returns self.

```

```

"""

def get_posterior_samples(self) -> np.ndarray:
    """
    Returns
    -----
    samples : np.ndarray, shape (n_total_samples, 3)
        Columns: [ $\phi$ ,  $\kappa_{\text{net}}$ ,  $\Delta t^*_{\text{low}}$ ]
    """

def posterior_mean_weight(
    self,
    dt: np.ndarray,
    delta_T: np.ndarray,
) -> np.ndarray:
    """
    Compute posterior mean weight  $\bar{w}(t) = E_{\psi}[\sigma(\phi(t; \psi))]$ .

    Averages  $\sigma(\phi(t; \psi^s))$  over posterior samples.
    NOTE: This averages  $\sigma(\phi)$  over samples, NOT  $\sigma$  of the average  $\phi$ .
    These differ by Jensen's inequality.

    Returns
    -----
    w_bar : np.ndarray, shape (n_proj,)
    """

def posterior_weight_ensemble(
    self,
    dt: np.ndarray,
    delta_T: np.ndarray,
    n_samples: int | None = None,
) -> np.ndarray:
    """
    Return weight trajectories for each posterior sample.

    Returns
    -----
    w_ensemble : np.ndarray, shape (n_psi_samples, n_proj)
    """

def convergence_diagnostics(self) -> dict:
    """
    MCMC convergence diagnostics.

     $\hat{R}$  is computed by splitting the post-warmup chain into 4 segments
    and treating each as an independent chain, per Vehtari et al. (2021).

```

This is necessary because emcee walkers are not independent.

Returns

-----

diagnostics : dict with keys:

"R\_hat" : np.ndarray, shape (3,) – split-chain  $\hat{R}$

"ESS" : np.ndarray, shape (3,) – effective sample size

"acceptance\_fraction" : float

"""

```
def holdout_influence_diagnostic(self, cv_data: list[dict]) -> dict:
```

"""

Leave-one-holdout-out influence diagnostic.

For each holdout endpoint  $t_h^{(j)}$ , re-runs MCMC with that holdout removed and reports the change in posterior medians.

Parameters

-----

cv\_data : list[dict] – the full CV data

Returns

-----

influence : dict with keys:

"holdout\_endpoints" : np.ndarray

"median\_shift\_phi0" : np.ndarray – shift per holdout

"median\_shift\_kappa\_net" : np.ndarray

"median\_shift\_dt\_low" : np.ndarray

"flagged" : list[int] – indices where any shift > 0.5 SD

"""

## Implementation notes

- The cross-validation requires refitting the rate-and-state model for each holdout endpoint. `model_refit_fn` is a callback the caller provides.
- For the trend agent density at each holdout, use the Gaussian predictive from the WLS posterior (analytically computed).
- For the model agent density, approximate as Gaussian with mean and std from the model posterior predictive ensemble. This is adequate for the synthesis calibration.
- The emcee sampler operates in transformed space:  $\log(\kappa_{\text{net}})$  and  $\log(\Delta t_{\text{low}})$  are sampled, with  $\phi_0$  sampled directly with a reflecting boundary at 0.
- The log-likelihood involves  $\log(w \cdot h_1 + (1-w) \cdot h_2)$ . Use the log-sum-exp trick to avoid



numerical underflow.

- The `prior_predictive_check` function is a prerequisite for `fit`. If checks fail, do not proceed.
  - The holdout influence diagnostic re-runs MCMC  $J$  times (once per holdout removed). This is computationally non-trivial but necessary.
- 

## Module 5: `diagnostics/trend_combination/combination.py`

**Purpose:** Generate the combined predictive distribution (mixture quantiles) and the temporally coherent stochastic-transition trajectory ensemble.

### Functions

```
def compute_mixture_quantiles(
    H_trend_ensemble: np.ndarray,
    H_model_ensemble: np.ndarray,
    w_bar: np.ndarray,
    quantiles: np.ndarray = np.array([0.05, 0.17, 0.50, 0.83, 0.95]),
) -> np.ndarray:
    """
    Compute quantiles of the BPS mixture predictive.

    The mixture distribution at each  $t$  is:
        
$$p_{\text{comb}}(H(t)) = \bar{w}(t) \cdot h_{\text{trend}}(H(t)) + (1 - \bar{w}(t)) \cdot h_{\text{model}}(H(t))$$


    Implementation: for each time step, pool trend and model samples,
    weight each trend sample by  $\bar{w}(t)/n_{\text{trend}}$  and each model sample by
     $(1-\bar{w}(t))/n_{\text{model}}$ , and compute weighted quantiles.

    Returns
    -----
    Q : np.ndarray, shape (n_quantiles, n_proj)
    """

def quantile_match_ensembles(
    H_trend_ensemble: np.ndarray,
    H_model_ensemble: np.ndarray,
    t_match_index: int,
) -> tuple[np.ndarray, np.ndarray]:
    """
    Sort and pair trend/model ensembles by rank at a reference time.
```

```

Parameters
-----
H_trend_ensemble : np.ndarray, shape (n_samples, n_proj)
H_model_ensemble : np.ndarray, shape (n_samples, n_proj)
t_match_index : int
    Index into the projection time array at which to sort.

Returns
-----
H_trend_matched : np.ndarray, shape (n_samples, n_proj)
H_model_matched : np.ndarray, shape (n_samples, n_proj)
    Ensembles reordered so that sample k in each corresponds
    to the k-th quantile at t_match.
"""

def generate_stochastic_transition_ensemble(
    H_trend_ensemble: np.ndarray,
    H_model_ensemble: np.ndarray,
    w_ensemble: np.ndarray,
    epsilon: float = 0.05,
    quantile_match: bool = True,
    t_match_index: int | None = None,
    seed: int | None = None,
) -> dict:
    """
    Generate temporally coherent trajectory ensemble via stochastic transition.

    For each MC sample s:
    1. Use  $w^{(s)}(t)$  from the posterior weight ensemble.
    2. Draw  $u^{(s)} \sim \text{Uniform}(0, 1)$ .
    3. Compute smooth transition indicator:
         $\omega^{(s)}(t) = \sigma((w^{(s)}(t) - u^{(s)}) / \epsilon)$ 
    4. Combine:
         $H_{\text{comb}}^{(s)}(t) = \omega^{(s)}(t) \cdot H_{\text{trend}}^{(s)}(t) + (1 - \omega^{(s)}(t)) \cdot H_{\text{model}}^{(s)}(t)$ 

    Parameters
    -----
    H_trend_ensemble : np.ndarray, shape (n_samples, n_proj)
    H_model_ensemble : np.ndarray, shape (n_samples, n_proj)
    w_ensemble : np.ndarray, shape (n_samples, n_proj)
    epsilon : float
        Sigmoid smoothing parameter.
    quantile_match : bool
        If True, apply quantile matching before combining.
    t_match_index : int or None

```

```

        Index for quantile matching. If None and quantile_match is True,
        use the index closest to the median transition time.
seed : int or None

Returns
-----
result : dict with keys:
    "H_combined" : np.ndarray, shape (n_samples, n_proj)
    "u_draws" : np.ndarray, shape (n_samples,)
    "transition_times" : np.ndarray, shape (n_samples,)
    "quantile_matched" : bool
"""

def marginal_fidelity_diagnostic(
    H_combined: np.ndarray,
    H_trend_ensemble: np.ndarray,
    H_model_ensemble: np.ndarray,
    w_bar: np.ndarray,
) -> dict:
    """
    Compare trajectory marginals to mixture marginals via KS test.

    For each time step, computes the weighted mixture CDF from the
    trend and model ensembles, and compares against the empirical
    CDF of the stochastic-transition ensemble.

    Returns
    -----
    diagnostic : dict with keys:
        "ks_statistic" : np.ndarray, shape (n_proj,)
        "max_ks" : float
        "times_exceeding_threshold" : np.ndarray
            Indices where KS > 0.05.
    """

def add_ais_component(
    H_comb_non_ais: np.ndarray,
    H_ais_dyn: np.ndarray,
) -> np.ndarray:
    """
    Add AIS dynamic component back to the combined non-AIS projection.

    
$$H_{\text{comb\_total}}^{(s)}(t) = H_{\text{comb\_non\_ais}}^{(s)}(t) + H_{\text{ais\_dyn}}^{(s)}(t)$$

    """

```

```

def rate_smoothness_diagnostic(
    H_combined: np.ndarray,
    t_proj: np.ndarray,
    epsilon_values: list[float] | None = None,
) -> dict:
    """
    Verify temporal coherence of the combined trajectory ensemble.

    Computes rates by finite differences and checks for discontinuities.
    Reports results for multiple epsilon values if provided.

    Parameters
    -----
    H_combined : np.ndarray, shape (n_samples, n_proj)
    t_proj : np.ndarray, shape (n_proj,)
    epsilon_values : list[float] or None
        If provided, report smoothness for each epsilon.
        Default: [0.02, 0.05, 0.10].

    Returns
    -----
    diagnostic : dict with keys:
        "max_rate_jump" : float
        "mean_rate_jump" : float
        "fraction_exceeding_threshold" : float
        Fraction of samples with any rate jump exceeding 1 mm/yr2
    """

```

## Implementation notes

- **Why stochastic transition, not fixed-weight linear combination.** The fixed-weight linear combination produces ensemble spread  $w^2 \cdot \text{Var}_{\text{trend}} + (1-w)^2 \cdot \text{Var}_{\text{model}}$  — always narrower than either. The stochastic-transition approach distributes trajectories between regimes, producing spread that includes inter-model variance.
- **Approximate marginal correctness.** The marginal correctness of the stochastic-transition ensemble is exact only as  $\epsilon \rightarrow 0$ . For finite  $\epsilon$  (default 0.05), samples in the transition region are convex combinations of matched trend and model values, producing distributions slightly narrower than the true mixture. The `marginal_fidelity_diagnostic` quantifies this approximation error.
- **Quantile matching tradeoff.** Quantile matching reduces rate jumps but introduces dependence between trend and model samples. The diagnostic should report results both with and without matching.

- **Matching ensemble sizes.** The trend and model ensembles must have the same `n_samples`. Resample with replacement before calling if sizes differ. Similarly for the weight ensemble.

## Module 6: `diagnostics/trend_combination/crossover_diagnostics.py`

**Purpose:** Compute crossover diagnostics from the posterior on synthesis hyperparameters.

### Functions

```
def compute_crossover_diagnostics(
    bps: "BPSSynthesis",
    t_proj: np.ndarray,
    t_end: float,
    delta_T_by_ssp: dict[str, np.ndarray],
    T_rate_by_ssp: dict[str, float] | None = None,
) -> dict:
    """
    Compute crossover diagnostics from the BPS posterior.

    For each SSP and each posterior sample  $\psi^s$ , the crossover  $t^*(s)$  is
    computed numerically by root-finding on  $\phi(t; \psi^s) = 0$ .

    The analytic approximation (closed-form) is:
        
$$\Delta t^* = (\phi_0 / (\kappa_{\text{net}} \cdot dT/dt^2 + \phi_0 / (\Delta t^*_{\text{low}})^4))^{1/4}$$


    Parameters
    -----
    bps : BPSSynthesis — fitted BPS object with posterior samples
    t_proj : np.ndarray, shape (n_proj,)
    t_end : float
    delta_T_by_ssp : dict mapping SSP name to  $\Delta T$  array
    T_rate_by_ssp : dict mapping SSP name to  $dT/dt$  (for analytic approx)

    Returns
    -----
    diagnostics : dict with keys:
        "crossover_posterior" : dict[str, np.ndarray]
        "crossover_median" : dict[str, float]
        "crossover_90CI" : dict[str, tuple[float, float]]
        "w_bar_at_2050" : dict[str, float]
        "w_bar_at_2100" : dict[str, float]
        "w_bar_timeseries" : dict[str, np.ndarray]
        "crossover_driver" : dict[str, str]
```

```

        "forcing_departure" or "acceleration_persistence" or "both"
    "analytic_crossover" : dict[str, float | None]
"""

def forcing_form_sensitivity(
    bps: "BPSSynthesis",
    t_proj: np.ndarray,
    t_end: float,
    delta_T_by_ssp: dict[str, np.ndarray],
) -> dict:
    """
    Compare crossover diagnostics under default ( $\Delta T^2 \cdot dt^2$ ) and
    optimistic ( $\Delta T^4 \cdot dt^2$ ) forcing-departure scaling.

    Returns
    -----
    sensitivity : dict with keys:
        "default_crossover_median" : dict[str, float]
        "optimistic_crossover_median" : dict[str, float]
        "difference_years" : dict[str, float]
        "exceeds_10yr" : bool
    """

```

---

## Module 7: Data Loading

### Satellite-era non-AIS GMSL

The WLS trend model is fit to satellite-era non-AIS GMSL:

$$\text{GMSL}_{\text{non\_AIS}}(t) = \text{GMSL}_{\text{total}}(t) - \text{GMSL}_{\text{AIS}}(t)$$

with AIS subtraction uncertainty propagated via  $\sigma_{\text{eff}}$  (see Module 2).

**Consistency requirement:** Use the same GMSL product for the WLS fit as for the rate-and-state calibration, subsetting to the satellite era.

### SSP temperature trajectories

Already available from Level 0. The same  $T_{\text{SSP}}(t)$  arrays used for the rate-and-state projection are passed to `compute_delta_T`.

---

# Execution Workflow

```
# =====
# Step 0: Prerequisites (already done)
# =====
# - Rate-and-state model calibrated, posterior samples available
# - Projection ensemble H_model[s, t] generated for each SSP
# - AIS dynamic component H_ais_dyn[s, t] separated

H_model_non_ais = {ssp: H_model[ssp] - H_ais_dyn[ssp] for ssp in ssps}

# =====
# Step 1: Calibration-domain temperature
# =====
T_max_cal = compute_T_cal_max(T_obs_calibration)

# =====
# Step 2: Fit WLS quadratic trend model to satellite-era non-AIS GMSL
# =====
wls = WLSTrendModel(satellite_start=1993.0, sigma_gia=0.15, fix_scale=True)
wls.fit(
    years_obs, gmsl_non_ais_obs, gmsl_uncertainty_obs,
    ais_subtraction_uncertainty=ais_sub_unc_obs, # time-dependent
)
robustness = wls.robustness_checks(
    years_obs, gmsl_non_ais_obs, gmsl_uncertainty_obs,
    ais_sub_unc_obs, t_end,
)
derivatives = wls.sample_derivatives(t_end=t_end, n_samples=N_SAMPLES)

# =====
# Step 3: Prior predictive check (MANDATORY)
# =====
delta_T_prior_check = {
    ssp: compute_delta_T(T_scenario[ssp], T_max_cal) for ssp in ssps
}
dt = t_proj - t_end

prior_check = prior_predictive_check(delta_T_prior_check, dt, n_prior_samples=100)
assert prior_check["checks_passed"], "Prior predictive check failed – revise prio

# =====
# Step 4: BPS cross-validation and posterior sampling
# =====
bps = BPSSynthesis(n_walkers=16, n_warmup=2000, n_samples=5000, seed=42)
```

```

cv_data = bps.build_cv_data(
    years=years_obs,
    gmsl=gmsl_non_ais_obs,
    gmsl_uncertainty=gmsl_uncertainty_obs,
    ais_subtraction_uncertainty=ais_sub_unc_obs,
    T_obs=T_obs,
    model_refit_fn=model_refit_fn,
)

bps.fit(cv_data)
convergence = bps.convergence_diagnostics()
assert all(convergence["R_hat"] < 1.05), "MCMC did not converge"
assert all(convergence["ESS"] > 400), "Insufficient ESS"

# =====
# Step 5: Holdout influence diagnostic (NON-OPTIONAL)
# =====
influence = bps.holdout_influence_diagnostic(cv_data)
if influence["flagged"]:
    print(f"WARNING: Holdout(s) {influence['flagged']} have outsized influence")

# =====
# Step 6: For each SSP
# =====
results = {}
for ssp in ssps:
    delta_T = compute_delta_T(T_scenario[ssp], T_max_cal, overshoot_mode=True)

    # 6a: Trend projection ensemble
    trend_result = trend_projection_ensemble(t_proj, t_end, derivatives)

    # 6b: Posterior mean weight
    w_bar = bps.posterior_mean_weight(dt, delta_T)

    # 6c: Mixture predictive quantiles (PRIMARY uncertainty reporting)
    Q_mixture = compute_mixture_quantiles(
        trend_result["H_ensemble"],
        H_model_non_ais[ssp],
        w_bar,
    )

    # 6d: Per-sample weight trajectories
    w_ensemble = bps.posterior_weight_ensemble(dt, delta_T, n_samples=N_SAMPLES)

    # 6e: Stochastic-transition trajectory ensemble (with quantile matching)
    transition_result = generate_stochastic_transition_ensemble(
        trend_result["H_ensemble"],

```



```

        H_model_non_ais[ssp],
        w_ensemble,
        epsilon=0.05,
        quantile_match=True,
    )

    # 6f: Marginal fidelity diagnostic
    fidelity = marginal_fidelity_diagnostic(
        transition_result["H_combined"],
        trend_result["H_ensemble"],
        H_model_non_ais[ssp],
        w_bar,
    )

    # 6g: Add AIS component back
    H_comb_total = add_ais_component(
        transition_result["H_combined"],
        H_ais_dyn[ssp],
    )

    # 6h: Rate smoothness diagnostic
    smoothness = rate_smoothness_diagnostic(H_comb_total, t_proj)

    results[ssp] = {
        "H_combined_non_ais": transition_result["H_combined"],
        "H_combined_total": H_comb_total,
        "Q_mixture_non_ais": Q_mixture,
        "w_bar": w_bar,
        "transition_times": transition_result["transition_times"],
        "smoothness": smoothness,
        "marginal_fidelity": fidelity,
    }

    # =====
    # Step 7: Crossover diagnostics
    # =====
    delta_T_by_ssp = {
        ssp: compute_delta_T(T_scenario[ssp], T_max_cal) for ssp in ssps
    }
    crossover = compute_crossover_diagnostics(bps, t_proj, t_end, delta_T_by_ssp)

    # =====
    # Step 8: Forcing-form sensitivity diagnostic
    # =====
    sensitivity = forcing_form_sensitivity(bps, t_proj, t_end, delta_T_by_ssp)

```

---

# Diagnostic Outputs

**Table: Crossover diagnostics**

| SSP          | t*<br>median<br>(yr) | t*<br>90%<br>CI | $\Delta t^*_{\text{low}}$<br>posterior<br>median (yr) | $\bar{w}(2050)$ | $\bar{w}(2100)$ | Driver         |
|--------------|----------------------|-----------------|-------------------------------------------------------|-----------------|-----------------|----------------|
| SSP1-<br>2.6 | ...                  | [...,<br>...]   | ...                                                   | ...             | ...             | accel_persist  |
| SSP2-<br>4.5 | ...                  | [...,<br>...]   | ...                                                   | ...             | ...             | both           |
| SSP3-<br>7.0 | ...                  | [...,<br>...]   | ...                                                   | ...             | ...             | forcing_depart |
| SSP5-<br>8.5 | ...                  | [...,<br>...]   | ...                                                   | ...             | ...             | forcing_depart |

**Table: BPS posterior summary**

| Parameter                 | Prior<br>median | Posterior<br>median | Posterior 90%<br>CI | Data-informed?           |
|---------------------------|-----------------|---------------------|---------------------|--------------------------|
| $\phi_0$                  | 2.5             | ...                 | [..., ...]          | Yes (well<br>identified) |
| $\kappa_{\text{net}}$     | 0.05            | ...                 | [..., ...]          | Weakly                   |
| $\Delta t^*_{\text{low}}$ | 20 yr           | ...                 | [..., ...]          | Moderately               |

**Table: Holdout influence**

| Holdout<br>t <sub>h</sub> | $\Delta$ median $\phi_0$ (in<br>SD) | $\Delta$ median $\kappa_{\text{net}}$ (in<br>SD) | $\Delta$ median $\Delta t^*_{\text{low}}$<br>(in SD) | Flagged? |
|---------------------------|-------------------------------------|--------------------------------------------------|------------------------------------------------------|----------|
| 2003                      | ...                                 | ...                                              | ...                                                  | ...      |
| 2005                      | ...                                 | ...                                              | ...                                                  | ...      |
| ...                       | ...                                 | ...                                              | ...                                                  | ...      |

## Figures

1. **Prior predictive weight trajectories.** Plot  $w(t)$  for 100 prior samples for each SSP. Verify reasonable prior behavior before CV.
2. **Posterior weight timeseries.** Plot  $\bar{w}(t)$  vs year for all SSPs. Shaded: 5th–95th percentile of  $w(t; \psi^{\wedge}(s))$ .
3. **Combined projection fan chart.** For SSP2–4.5: model-only, trend-only, BPS mixture, and stochastic-transition CIs with crossover region.
4. **BPS posterior.** Corner plot for  $(\phi_0, \kappa_{\text{net}}, \Delta t^*_{\text{low}})$ .
5. **Cross-validation scores.** Per-holdout log predictive score as a function of lead time.
6. **Transition time distribution.** Histogram per SSP.
7. **Rate smoothness.** Overlay 50 trajectories and their rates. Compare with and without quantile matching.
8. **Marginal fidelity.** KS statistic vs time for the stochastic-transition ensemble.
9. **Forcing-form sensitivity.** Posterior median crossover under default vs optimistic  $\Delta T$  scaling.

---

## Parameter Choices

| Parameter                        | Symbol                | Value         | Justification                     |
|----------------------------------|-----------------------|---------------|-----------------------------------|
| Trend model                      | —                     | WLS quadratic | Closed-form, zero hyperparameters |
| Satellite start                  | —                     | 1993.0        | Start of TOPEX/Poseidon altimetry |
| GIA uncertainty                  | $\sigma_{\text{GIA}}$ | 0.15 mm/yr    | Half the inter-model GIA spread   |
| AIS subtraction unc. (pre-GRACE) | $\sigma_{\text{AIS}}$ | ~0.5 mm/yr    | IMBIE reconciliation spread       |
| AIS subtraction unc. (GRACE era) | $\sigma_{\text{AIS}}$ | ~0.15 mm/yr   | GraMBIE uncertainty               |
| Residual scale                   | $\hat{\sigma}^2$      | 1.0 (fixed)   | Trusted observation               |

|                                 |   |                                             |                                               |
|---------------------------------|---|---------------------------------------------|-----------------------------------------------|
|                                 |   |                                             | uncertainties                                 |
| $\phi_0$ prior                  | — | $N(2.5, 0.5^2), \phi_0 > 0$                 | Trend precision advantage at $t_{\text{end}}$ |
| $\kappa_{\text{net}}$ prior     | — | $\text{LogNormal}(\log(0.05), 0.6)$         | See prior predictive check                    |
| $\Delta t^*_{\text{low}}$ prior | — | $\text{LogNormal}(\log(20), 0.35)$          | ~20 yr under constant T; 95% CI [10, 38]      |
| Overshoot mode                  | — | True                                        | Conservative for irreversible processes       |
| n_samples                       | — | 10000                                       | Match rate-and-state ensemble size            |
| MCMC walkers                    | — | 16                                          | emcee default for 3-parameter problem         |
| MCMC warmup                     | — | 2000                                        | Conservative; check $\hat{R}$                 |
| MCMC samples                    | — | 5000                                        | Per walker; total 80,000, thinned to ~8,000   |
| Smoothing $\varepsilon$         | — | 0.05                                        | Transition over ~2–5 years                    |
| CV holdout endpoints            | — | $\{2003, 2005, \dots, t_{\text{end}} - 5\}$ | 2-year spacing, min 5 yr held out             |

## Testing Strategy

### Unit tests (per module)

Each module has its own test file. Tests should cover:

1. **Deterministic correctness:** Given known inputs, check outputs against hand-calculated values.
2. **Edge cases:**  $\Delta T = 0$  everywhere,  $\Delta T = \text{constant}$ , single-point WLS fit (should raise error).
3. **Dimensional consistency:** All rates in mm/yr.
4. **Weight bounds:**  $w(t)$  is always in  $[0, 1]$ , including at extreme parameter values.

5. **Temporal coherence:** Rates on combined trajectories are smooth ( $\text{max\_rate\_jump}$  below threshold).
6. **WLS covariance propagation:** Jacobian-propagated covariance matches reparametrised fit.
7. **GIA inflation:** Only rate variance is inflated, not acceleration variance.
8. **AIS subtraction uncertainty:** Propagated correctly into WLS weights.
9. **Reparameterization consistency:**  $\kappa_t$  derived from  $(\phi_0, \Delta t^*_{\text{low}})$  produces the same logit as direct  $\kappa_t$  parameterization.
10. **Crossover under zero  $\Delta T$ :** Crossover time equals  $\Delta t^*_{\text{low}}$  exactly.
11. **Synthesis log-likelihood:** On synthetic CV data where the trend is clearly better at short leads, verify  $\phi_0$  posterior is large and positive.
12. **Mixture quantiles:** Mixture 50th percentile lies between trend and model medians.
13. **Stochastic transition marginal fidelity:** KS statistic between trajectory marginals and mixture marginals is below 0.05 at each  $t$ , within the  $\varepsilon$ -approximation tolerance.
14. **AIS add-back:**  $H_{\text{comb\_total}} = H_{\text{comb\_non\_ais}} + H_{\text{ais\_dyn}}$  exactly.
15. **Quantile matching:** With quantile matching, max rate jump is reduced relative to without matching. Marginal distributions of each ensemble are unchanged.
16. **Prior predictive check:** Verify that the default priors pass the four checks.

## Integration test

End-to-end test with synthetic data:

1. Generate synthetic GMSL from a known rate-and-state model.
2. Generate synthetic AIS dynamic contribution.
3. Add realistic noise.
4. Fit WLS to the non-AIS satellite-era data (with AIS subtraction uncertainty).
5. Fit the rate-and-state model.
6. Run prior predictive check.
7. Run BPS cross-validation and posterior sampling.
8. Generate combined projections.

## 9. Verify:

- The mixture predictive matches the trend at short leads ( $\bar{w} \approx 1$ ).
- The mixture predictive matches the model at long leads ( $\bar{w} \approx 0$ ).
- Under a “flat temperature” scenario ( $\Delta T = 0$ ), the crossover is near  $\Delta t^*_{\text{low}}$ .
- Under a “rapid warming” scenario, the crossover occurs within  $\sim 10$  years.
- Individual stochastic-transition trajectories are smooth.
- The AIS component passes through unmodified.
- The mixture quantile CI is wider than the stochastic-transition ensemble CI (expected consequence of finite  $\epsilon$ ).
- The KS diagnostic quantifies this discrepancy.

## Regression test

Snapshot diagnostics and posterior summary. Check future changes do not alter results beyond MCMC sampling noise.

---

## Open Questions

1. **Cross-validation cost.** The leave-future-out CV requires refitting the rate-and-state model for each holdout endpoint ( $\sim 8$ – $10$  refits). The holdout influence diagnostic adds another  $\sim 8$ – $10$  MCMC runs. If full MCMC is too expensive, a Laplace approximation can be used for the CV refits only. The sensitivity of the BPS posterior to the CV model approximation should be checked.
2. **Non-Gaussian agent densities.** The current implementation approximates the model agent density as Gaussian in the CV step. A diagnostic should compare the CV log-likelihood under Gaussian vs. KDE of the model predictive.
3. **BPS with  $>2$  agents.** The framework naturally extends to  $K > 2$  agents. The current 2-agent implementation should accommodate this extension.
4. **emcee multimodality.** The log-likelihood is a sum of log-mixture terms, which can be multimodal. A pilot run should check for multimodality before committing to emcee. If multimodal, consider nested sampling (e.g., `dynesty`) instead.